CO1 ASSIGNMENT-1

TASK 1 – Understand the Problem and Identify the Data

1. Inputs

The electricity billing program requires the following inputs:

1. Previous reading – The meter reading recorded during the previous month.
2. Current reading – The latest meter reading.
3. Household number – A unique number used to identify each household.
4. Category – The type of household.

These inputs are used to calculate the electricity consumption and prepare the bill.

2. Outputs

The program produces the following outputs:

1. Units consumed – Calculated by subtracting the previous reading from the current reading.
2. Energy charge – Calculated based on the number of units consumed and the tariff slabs.
3. Fixed charge – A fixed amount of Rs. 50 added to the bill.
4. Total bill – The sum of the energy charge and fixed charge.

3. Constants

Constants are values that do not change during program execution. The following values are used in this program:

- First slab limit = 100 units.
- Second slab limit = 200 units.
- First slab rate = Rs. 0 per unit.
- Second slab rate = Rs. 2.25 per unit.
- Third slab rate = Rs. 4.50 per unit.
- Fixed charge = Rs. 50.

Using constants makes the program easier to maintain because tariff changes can be made in one place.

4. Values That Can Be Negative, Fractional or Very Large

- Negative values: A bill should never be negative. However, the current meter reading may be smaller than the previous reading when the meter rolls over.
- Fractional values: Electricity charges may contain paise. For example, Rs. 2.25 per unit.
- Very large values: The number of households may increase from 4,000 to 800,000 at district level. Therefore, the program must handle large values.

5. Rules for a Correct Bill

The following rules must be followed:

1. The electricity bill should never be negative.
2. Units consumed must be calculated correctly.
3. The first 100 units are free.
4. Units from 101 to 200 are charged at Rs. 2.25 per unit.

5. Units above 200 are charged at Rs. 4.50 per unit.
6. The total bill must be equal to the fixed charge plus the energy charge.
7. Meter rollover must be handled correctly.

6. Assumptions

1. Each household has a unique household number.
2. The meter uses a 16-bit unsigned counter.
3. The maximum meter reading is 65535.
4. After reaching 65535, the meter resets to 0.
5. A fixed charge of Rs. 50 is applied to every household.
6. The bill is calculated once every month.
7. The tariff rates remain unchanged during one program execution.

7. Alternative Concept and Complexity Analysis

If the meter readings are stored as character strings instead of integers, the readings must be converted into numbers before performing calculations.

For example, the reading 1530 can be stored as "01530".

In string representation, each digit is stored as a character. Converting a string into an integer requires checking each digit. Therefore, the time required is O(d), where d is the number of digits.

In integer representation, calculations and comparisons can be performed directly. Integer storage also requires a fixed

amount of memory, whereas string storage depends on the number of digits.

Comparison:

Integer Representation	String Representation
Stores readings as numbers.	Stores readings as characters.
Calculations are easy.	Conversion is required before calculations.
Uses fixed memory.	Memory depends on the number of digits.
Faster for arithmetic operations.	Requires extra processing.

Conclusion:

Integer representation is more suitable for the electricity billing program because it makes calculations easier and faster. It also uses fixed memory for storing meter readings.

TASK 2 – Choose the Data Types

1. Selection of Data Types

Different data types are used in the billing program depending on the type and size of the values.

Data Item	Data Type	Reason
Previous reading	unsigned short	Stores meter readings from 0 to 65535.

Current reading	unsigned short	Stores the current meter reading.
Household number	int	Stores the unique household number.
Category	char	Stores a single category character.
Units consumed	unsigned int	Stores the number of units consumed.
Energy charge	long	Stores the energy charge in paise.
Fixed charge	long	Stores the fixed charge in paise.
Total bill	long	Stores the total bill in paise.

2. Size and Range of Data Types

The size of data types can be checked using the sizeof operator. The limits are available in the limits.h and float.h header files.

Data Type	**Size (Typical)**	**Range**
char	1 byte	-128 to 127
short	2 bytes	-32768 to 32767
unsigned short	2 bytes	0 to 65535

int	4 bytes	-2147483648 to 2147483647
unsigned int	4 bytes	0 to 4294967295
long	8 bytes	Depends on system
long long	8 bytes	Approximately -9.22×10^{18} to 9.22×10^{18}
float	4 bytes	Approximately 6–7 decimal digits of precision
double	8 bytes	Approximately 15–16 decimal digits of precision

The selected data types are sufficient to handle the billing calculations at district level.

3. Why Money Should Not Be Stored as Float

Money should not be stored as float because float cannot represent some decimal values exactly. This may produce small rounding errors during calculations.

For example, Rs. 2.25 may not be stored exactly in float.

Instead, money can be stored as integer paise.

Example:

Rs. 162.50 = 16250 paise.

This method avoids floating-point rounding errors and gives accurate billing results.

4. Example of Float Error for 4000 Bills

If each bill contains Rs. 162.25, the total for 4000 households is:

Total = 162.25 × 4000

Total = Rs. 649000.00

When float is used, small precision errors may occur while adding many bills. These errors can affect the final amount.

By storing the amount in integer paise:

16225 × 4000 = 64900000 paise.

Therefore, the total is exactly Rs. 649000.00.

5. What Happens to a 16-bit Unsigned Meter Register After 65535?

A 16-bit unsigned meter register can store values from 0 to 65535.

After reaching 65535, it resets to 0. This is called meter rollover.

For example:

Previous reading = 65400

Current reading = 120

Units consumed = 256 units.

The calculation is:

(65535 - 65400) + 1 + 120 = 256 units.

6. Complete the Code – TODO 1

The following C program calculates the units consumed correctly even when the meter reading rolls over.

```
#include <stdio.h>
#include <limits.h>

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    return (meter_t)(current - previous);
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);

    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120));
```

```
    return 0;
}
```

Output:

sizeof(meter_t) = 2 bytes, maximum reading = 65535

Consumption: 330 units

Consumption: 256 units

7. Explanation of the Second Result

The second result is 256 because the meter reading has rolled over from 65535 to 0.

The calculation is:

(65535 - 65400) + 1 + 120 = 256.

Since unsigned 16-bit arithmetic wraps around after 65535, the result is 256 instead of a negative number.

8. Alternative Concept and Complexity Analysis

If money is stored as double rupees instead of long paise, the program can perform calculations using decimal values.

However, double may also produce small precision errors because some decimal values cannot be represented exactly in binary.

Rounding can be applied when the final bill is calculated, before displaying the amount.

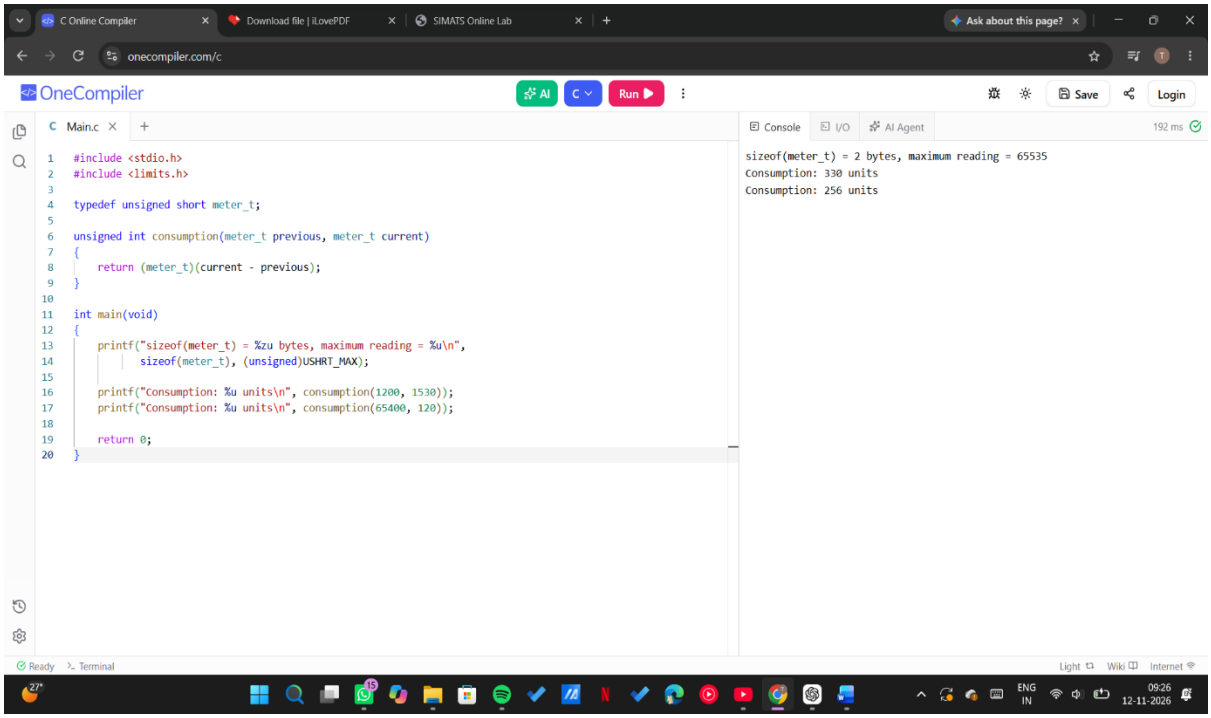
Comparison:

Integer Paise	Double Rupees
Stores money as whole paise.	Stores money as decimal rupees.
Avoids floating-point rounding errors.	May produce small precision errors.
Requires 8 bytes when using long on a typical 64-bit Linux system.	Usually requires 8 bytes.
Addition takes O(1) time.	Addition takes O(1) time.

When n bills are added using double, small rounding errors may accumulate. Integer paise avoids this problem as long as the integer type does not overflow.

Conclusion:

Integer paise using long is a better choice for the electricity billing program because it provides accurate money calculations. Unsigned short is suitable for meter readings because the meter has a maximum value of 65535.



TASK 3 – Define the Constants

1. Introduction

Constants are fixed values that do not change during program execution. In the electricity billing program, tariff rates, slab limits and fixed charges are defined as constants.

Using constants makes the program easier to maintain. If the electricity tariff changes, we only need to modify the values in one place.

2. Use of #define, const and enum

1. #define

- It is a preprocessor directive.
- It is used to define fixed values.
- It replaces the defined name with its value before compilation.

- It does not require memory for storing the constant.

Example:

```
#define RATE2_PAISE 225

#define FIXED_CHARGE_PAISE 5000L
```

2. const

- It is used to declare a variable whose value cannot be changed.
- It has a specific data type.
- It provides type checking.

Example:

```
const int rate = 225;
```

3. enum

- It is used to define a group of named integer constants.
- It is useful for representing categories or fixed options.

Example:

```
enum category
{
   DOMESTIC,
   COMMERCIAL
};
```

3. Difference Between #define and const

#define	**const**

It is a preprocessor directive.	It is a C keyword.
It does not have a data type.	It has a data type.
It does not provide type checking.	It provides type checking.
It is replaced before compilation.	It is handled during compilation.
It is not directly visible as a variable in the debugger.	It can be easier to identify in the debugger.

4. Naming Conventions for Constants

1. Constant names should be meaningful.
2. Capital letters are generally used.
3. Underscores can be used to separate words.
4. Names should clearly describe their purpose.

Examples:

- SLAB1_UNITS
- SLAB2_UNITS
- RATE1_PAISE
- RATE2_PAISE
- RATE3_PAISE
- FIXED_CHARGE_PAISE

5. Complete the Code – TODO 1

The following C program calculates the electricity bill using macros. It calculates the energy charge according to the number of units consumed.

C Program:

```
#include <stdio.h>


#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L


long compute_bill(int units)
{
   long paise = FIXED_CHARGE_PAISE;


   if (units > SLAB2_UNITS)
   {
      paise += (units - SLAB2_UNITS) * RATE3_PAISE;
      units = SLAB2_UNITS;
   }
```

```
    if (units > SLAB1_UNITS)
    {
        paise += (units - SLAB1_UNITS) * RATE2_PAISE;
        units = SLAB1_UNITS;
    }

    paise += units * RATE1_PAISE;

    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof tests / sizeof tests[0];

    for (int i = 0; i < n; i++)
    {
        long p = compute_bill(tests[i]);

        printf("%3d units -> Rs %ld.%02ld\n",
```

```
            tests[i], p / 100, p % 100);
    }


    return 0;
}
```

Output:

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

6. Changing the Tariff Rate

If the second slab rate is changed from 225 paise to 230 paise, we only need to modify the macro:

```
#define RATE2_PAISE 230
```

The new output will be:

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 165.00
200 units -> Rs 280.00
350 units -> Rs 955.00
```

All bills are updated automatically because the tariff is defined in one place.

7. Alternative Concept and Complexity Analysis

If const variables are used instead of #define, the constants will have data types and type checking. They may also occupy memory.

If enum is used, it is suitable for fixed integer values such as household categories, but it is not suitable for storing decimal tariff values directly.

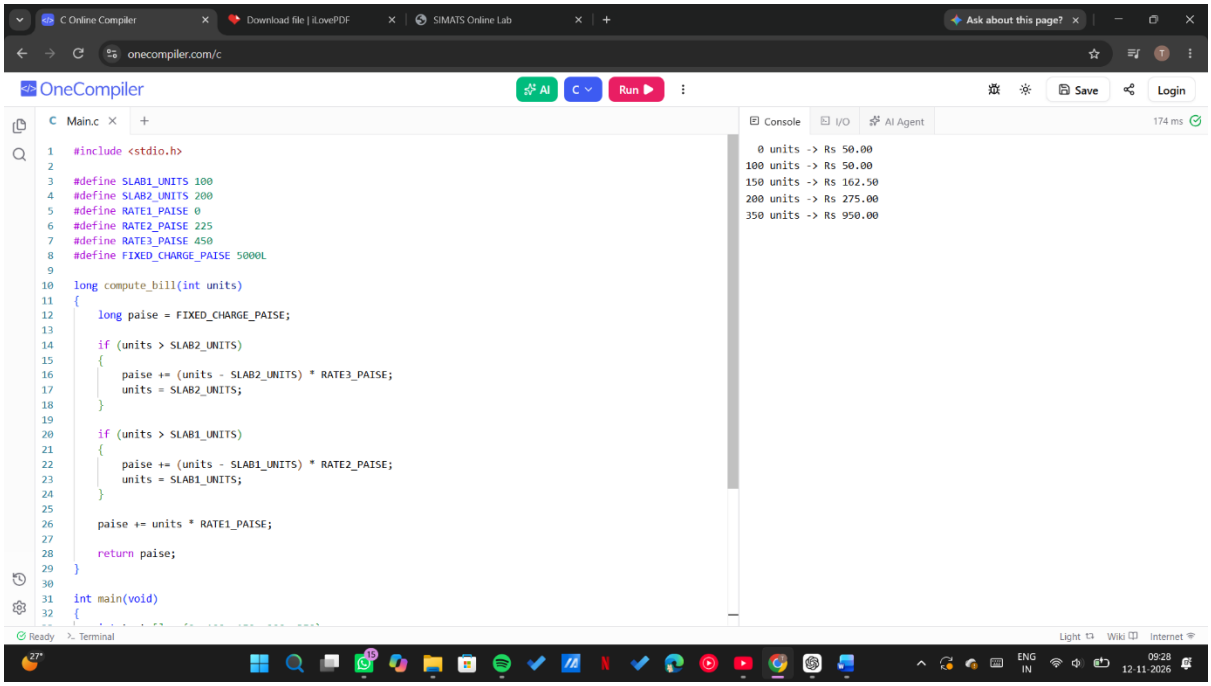
Comparison:

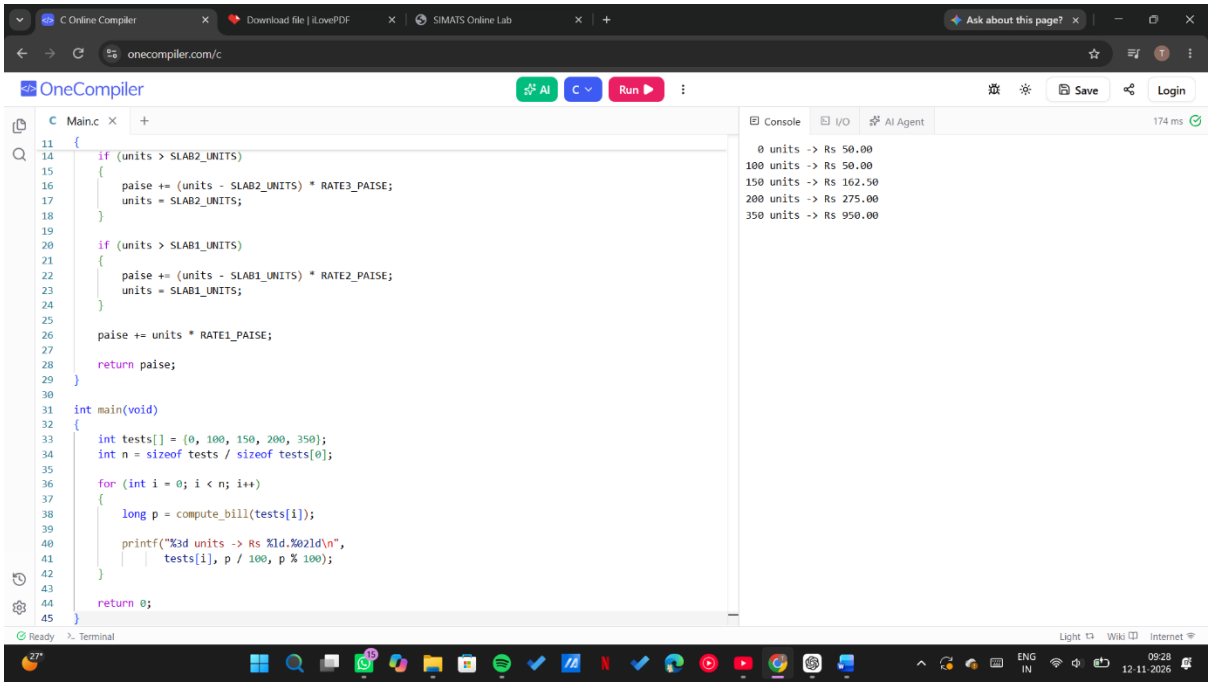
Feature	**#define**	**const**	**enum**
Type checking	No	Yes	Yes
Memory	No storage required	May occupy memory	No storage for the definition itself
Debugging	Less visible	Easier to identify	Names can be identified
Main use	Fixed values	Typed constants	Named integer constants

The running time is generally not affected because these values are replaced or accessed efficiently during compilation.

Conclusion:

I have used #define for tariff rates and slab limits because it allows tariff changes to be made in one place. It makes the billing program simple, consistent and easy to maintain.





TASK 4 – Use Macros Safely

1. Introduction

Macros are defined using the #define preprocessor directive in C. They are used to perform repeated calculations and reduce code repetition.

Macros are expanded before the program is compiled.

2. Examples of Macros

The following macros are used for simple calculations:

#define RUPEES_TO_PAISE(x) ((x) * 100)

#define SQUARE(x) ((x) * (x))

#define MAX(a,b) ((a) > (b) ? (a) : (b))

- RUPEES_TO_PAISE converts rupees into paise.
- SQUARE calculates the square of a number.
- MAX finds the larger of two numbers.

3. How a Macro Is Expanded

The preprocessor replaces the macro name with its defined expression before compilation.

For example:

#define SQUARE(x) ((x) * (x))

int a = 5;

int result = SQUARE(a);

After preprocessing, it becomes:

int a = 5;

int result = ((a) * (a));

We can see the expanded code using the command:

gcc -E program.c

4. Problems with Incorrect Macros

1. Problem with SQUARE(x)

If the macro is written without parentheses:

#define SQUARE(x) x*x

For the expression:

SQUARE(a+1)

It expands into:

a+1*a+1

If a = 2, the result is 5 instead of 9.

The correct macro is:

#define SQUARE(x) ((x)*(x))

Now it expands correctly:

((a+1)*(a+1))

2. Problem with MAX(a,b)

The MAX macro can evaluate an argument more than once.

For example:

#define MAX(a,b) ((a) > (b) ? (a) : (b))

```
int i = 5;
int result = MAX(i++, 10);
```

The expression i++ may be evaluated twice depending on the comparison result. This can cause unexpected changes in the value of i.

Therefore, macros containing increment or decrement operators should be avoided.

5. When a Function Is Better Than a Macro

A function is better than a macro when:

1. Type checking is required.
2. Arguments should not be evaluated multiple times.
3. The program should be easier to debug.
4. The calculation is complex.

Functions are safer and easier to maintain than macros for many calculations.

6. Alternative Concept and Complexity Analysis

If MAX and SQUARE macros are replaced by static inline functions, the program becomes safer.

Example:

```
static inline int square(int x)
{
   return x * x;
}


static inline int maximum(int a, int b)
```

```
{
   return (a > b) ? a : b;
}
```

Comparison:

Macros	Static Inline Functions
Expanded by the preprocessor.	Defined as normal functions.
No type checking.	Provides type checking.
Arguments may be evaluated more than once.	Arguments are evaluated once.
Can increase code size when used many times.	Compiler may avoid repeated function call overhead.
Difficult to debug.	Easier to debug.

Time Complexity:

Both macros and inline functions perform simple calculations in O(1) time.

Space Complexity:

If a macro is used k times, its expanded code is copied at every use, which may increase the program size.

An inline function may reduce repeated code, but the compiler decides whether to actually inline it.

7. Conclusion

Macros are useful for simple repeated calculations. Parentheses should always be used to avoid errors during macro expansion. However, static inline functions are a safer choice when type checking and debugging are important.

TASK 5 – Organise the Program with Header Files and Conditional Compilation

1. Introduction

A C program can be organised using header files, global declarations, the main() function and other functions.

Header files are used to store constants, macros and function declarations. This makes the program easier to understand and maintain.

2. Structure of a C Program

A C program generally contains the following parts:

1. Header files – Used to include the required library files.
2. Global declarations – Used to declare constants and variables.
3. Function prototypes – Used to declare functions before they are called.
4. main() function – The starting point of program execution.
5. User-defined functions – Used to perform specific tasks.

3. Include Guards

Include guards are used to prevent the same header file from being included more than once.

They are written using three preprocessor directives:

- #ifndef checks whether the macro is already defined.
- #define defines the macro.
- #endif ends the condition.

Example:

```
#ifndef BILLING_H
#define BILLING_H

#define RATE2_PAISE 225
#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units);

#endif
```

This prevents multiple inclusion of the same header file.

4. Conditional Compilation Using DEBUG

Conditional compilation allows us to include debugging statements only when required.

Example:

```
#ifdef DEBUG
```

printf("Calculating bill for %d units\n", units);

#endif

The debugging statement is included only when DEBUG is defined.

If DEBUG is not defined, the debugging statement is removed during preprocessing.

5. Difference Between #include <file> and #include "file"

#include <file>	**#include "file"**
Used for standard library header files.	Used for user-defined header files.
Searches standard system directories.	Searches the current directory first.
Example: stdio.h	Example: billing.h

6. Complete C Program

The following program combines the constants, billing function and main function in a single file. This makes it easier to compile and execute using OneCompiler.

Program:

```
#include <stdio.h>


#define DEBUG


#define SLAB1_UNITS 100
```

```
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units)
{
   long paise = FIXED_CHARGE_PAISE;

#ifdef DEBUG
   printf("Calculating bill for %d units\n", units);
#endif

   if (units > SLAB2_UNITS)
   {
      paise += (units - SLAB2_UNITS) * RATE3_PAISE;
      units = SLAB2_UNITS;
   }

   if (units > SLAB1_UNITS)
   {
```

```
        paise += (units - SLAB1_UNITS) * RATE2_PAISE;
        units = SLAB1_UNITS;
    }

    return paise;
}

int main()
{
    int units = 150;
    long bill = compute_bill(units);

    printf("Total Bill = Rs %ld.%02ld\n",
           bill / 100, bill % 100);

    return 0;
}
```

7. Output

```
Calculating bill for 150 units
Total Bill = Rs 162.50
```

8. Alternative Concept and Complexity Analysis

If debugging is controlled using a runtime variable instead of #ifdef DEBUG, an if condition is checked whenever the program executes the debugging statement.

Example:

```
int debug = 1;


if (debug)
{
   printf("Calculating bill\n");
}
```

Comparison:

#ifdef DEBUG	**Runtime Variable**
Checked during preprocessing.	Checked during program execution.
Debug code is removed when DEBUG is not defined.	Debug code remains in the executable.
No runtime condition checking is needed.	An extra condition is checked.
Saves executable space when disabled.	Uses additional executable space.

Time Complexity:

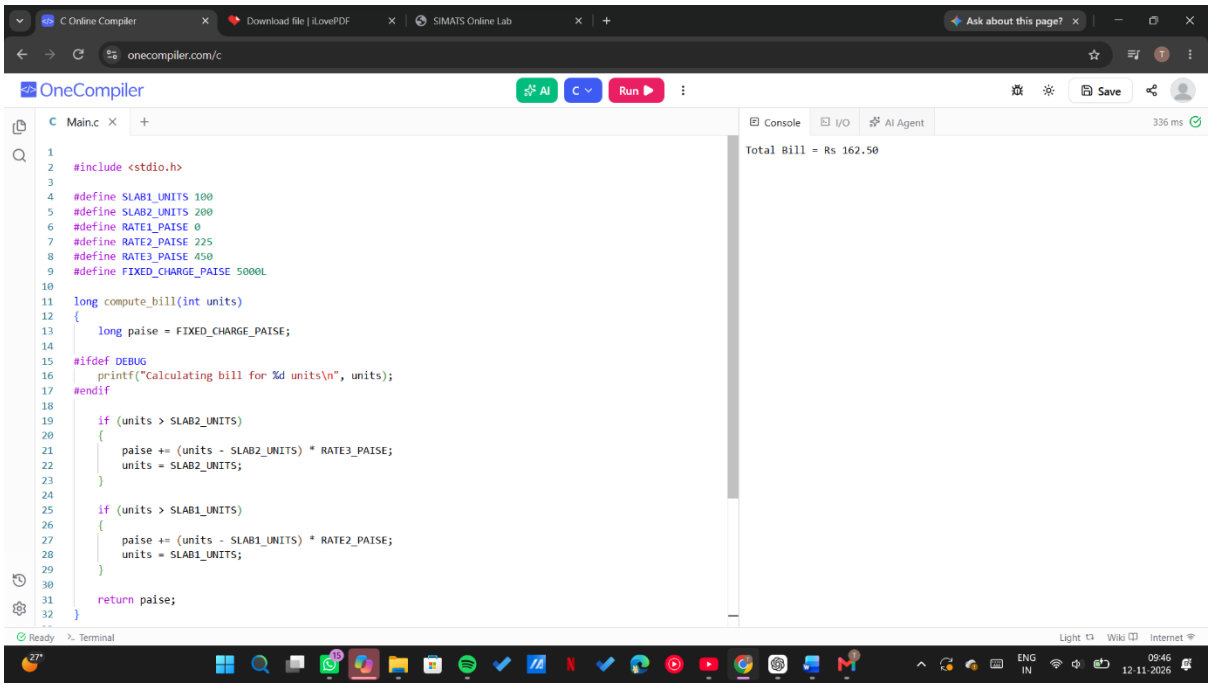
Both methods have O(1) time for checking the debugging condition. However, the runtime variable method performs an extra condition check during execution.

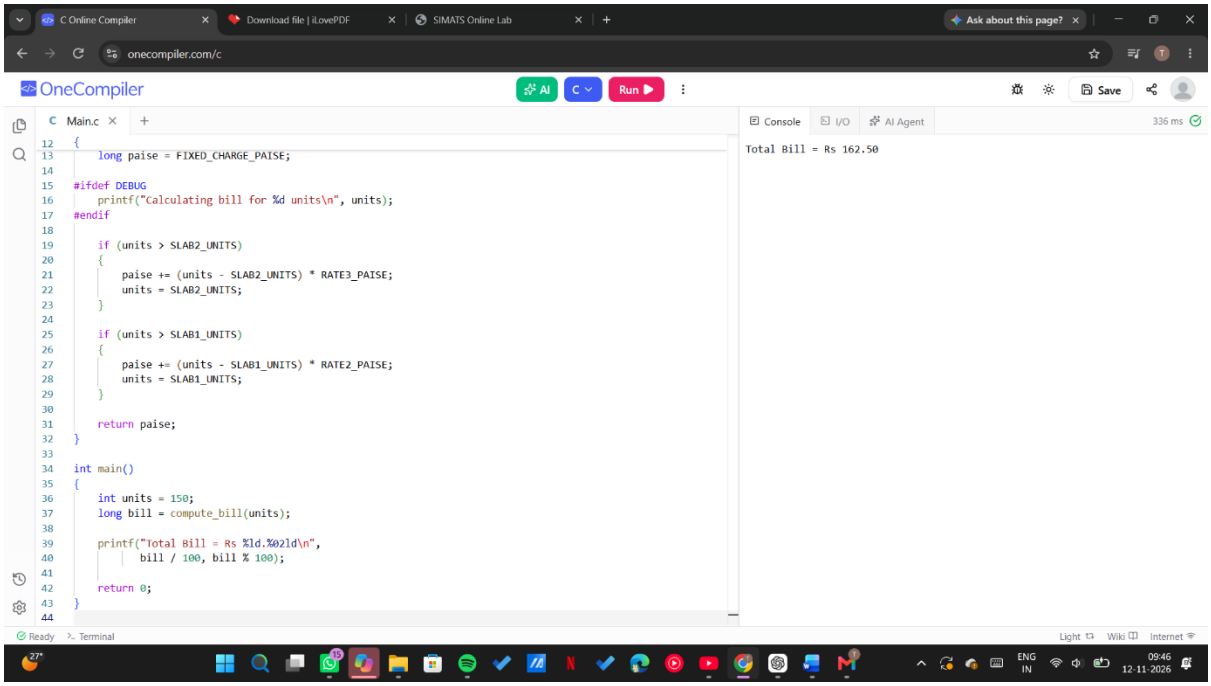
Space Complexity:

With #ifdef DEBUG, debugging code is not included when DEBUG is disabled. With a runtime variable, the debugging code remains in the executable.

9. Conclusion

Header files help organise the program and make it easier to maintain. Include guards prevent repeated inclusion of header files. Conditional compilation using #ifdef DEBUG helps display debugging information only when required. It also reduces unnecessary code in the final program.





TASK 5 – Organise the Program with Header Files and Conditional Compilation

1. Introduction

A C program can be organised using header files, global declarations, the main() function and other functions.

Header files are used to store constants, macros and function declarations. This makes the program easier to understand and maintain.

2. Structure of a C Program

A C program generally contains the following parts:

1. Header files – Used to include the required library files.
2. Global declarations – Used to declare constants and variables.

3. Function prototypes – Used to declare functions before they are called.
4. main() function – The starting point of program execution.
5. User-defined functions – Used to perform specific tasks.

3. Include Guards

Include guards are used to prevent the same header file from being included more than once.

They are written using three preprocessor directives:

- #ifndef checks whether the macro is already defined.
- #define defines the macro.
- #endif ends the condition.

Example:

```
#ifndef BILLING_H
#define BILLING_H


#define RATE2_PAISE 225
#define FIXED_CHARGE_PAISE 5000L


long compute_bill(int units);


#endif
```

This prevents multiple inclusion of the same header file.

4. Conditional Compilation Using DEBUG

Conditional compilation allows us to include debugging statements only when required.

Example:

```
#ifdef DEBUG

printf("Calculating bill for %d units\n", units);

#endif
```

The debugging statement is included only when DEBUG is defined.

If DEBUG is not defined, the debugging statement is removed during preprocessing.

5. Difference Between #include <file> and #include "file"

#include <file>	#include "file"
Used for standard library header files.	Used for user-defined header files.
Searches standard system directories.	Searches the current directory first.
Example: stdio.h	Example: billing.h

6. Complete C Program

The following program combines the constants, billing function and main function in a single file. This makes it easier to compile and execute using OneCompiler.

Program:

```
#include <stdio.h>

#define DEBUG

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;

#ifdef DEBUG
    printf("Calculating bill for %d units\n", units);
#endif

    if (units > SLAB2_UNITS)
    {
        paise += (units - SLAB2_UNITS) * RATE3_PAISE;
```

```
        units = SLAB2_UNITS;
    }

    if (units > SLAB1_UNITS)
    {
        paise += (units - SLAB1_UNITS) * RATE2_PAISE;
        units = SLAB1_UNITS;
    }

    return paise;
}

int main()
{
    int units = 150;
    long bill = compute_bill(units);

    printf("Total Bill = Rs %ld.%02ld\n",
           bill / 100, bill % 100);

    return 0;
}
```

7. Output

Calculating bill for 150 units

Total Bill = Rs 162.50

8. Alternative Concept and Complexity Analysis

If debugging is controlled using a runtime variable instead of #ifdef DEBUG, an if condition is checked whenever the program executes the debugging statement.

Example:

```
int debug = 1;


if (debug)
{
   printf("Calculating bill\n");
}
```

Comparison:

#ifdef DEBUG	**Runtime Variable**
Checked during preprocessing.	Checked during program execution.
Debug code is removed when DEBUG is not defined.	Debug code remains in the executable.
No runtime condition checking is needed.	An extra condition is checked.

Saves executable space when disabled.	Uses additional executable space.

Time Complexity:

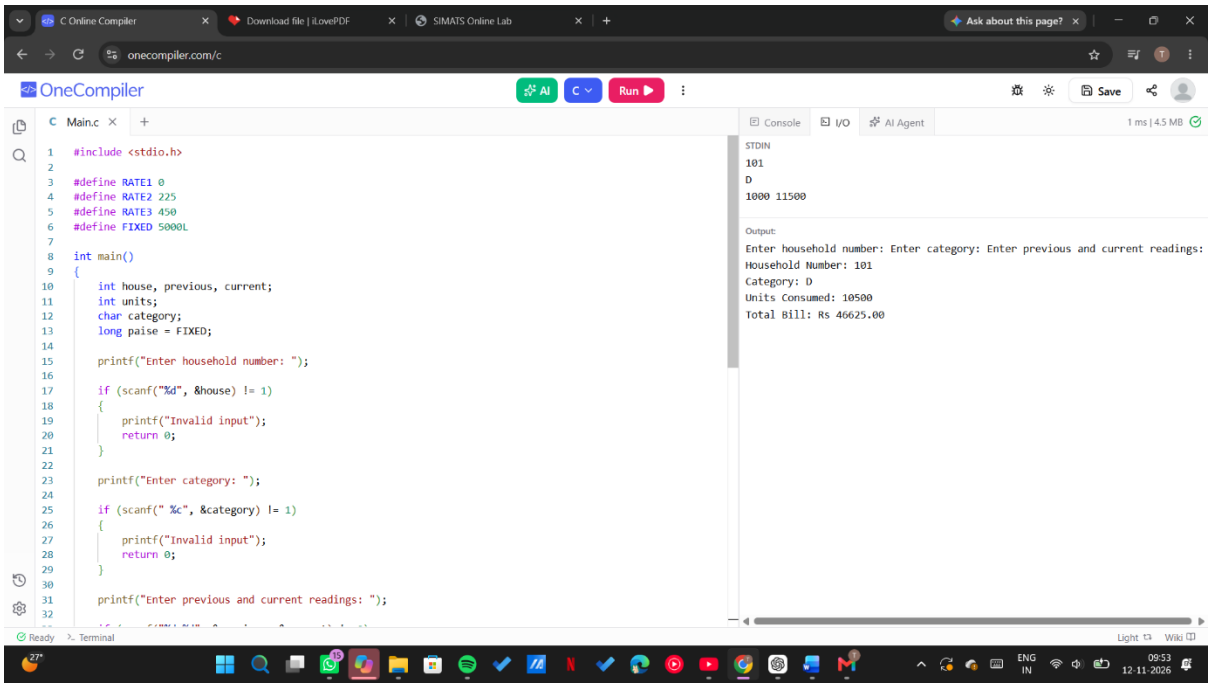
Both methods have O(1) time for checking the debugging condition. However, the runtime variable method performs an extra condition check during execution.

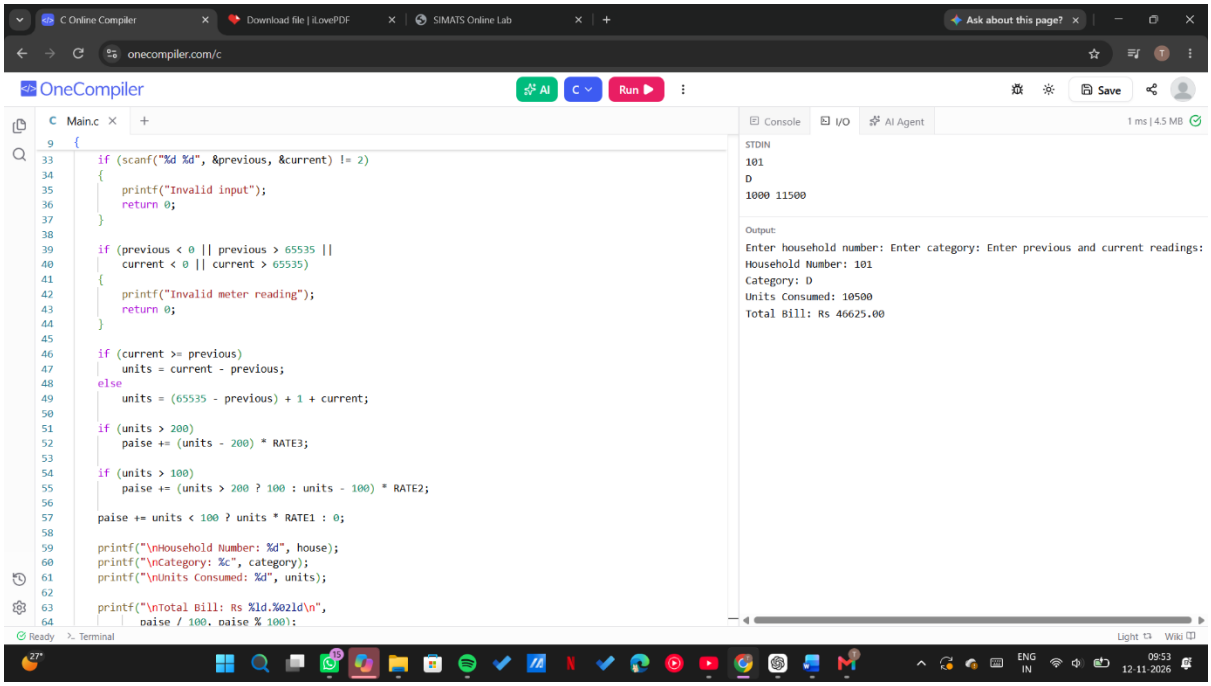
Space Complexity:

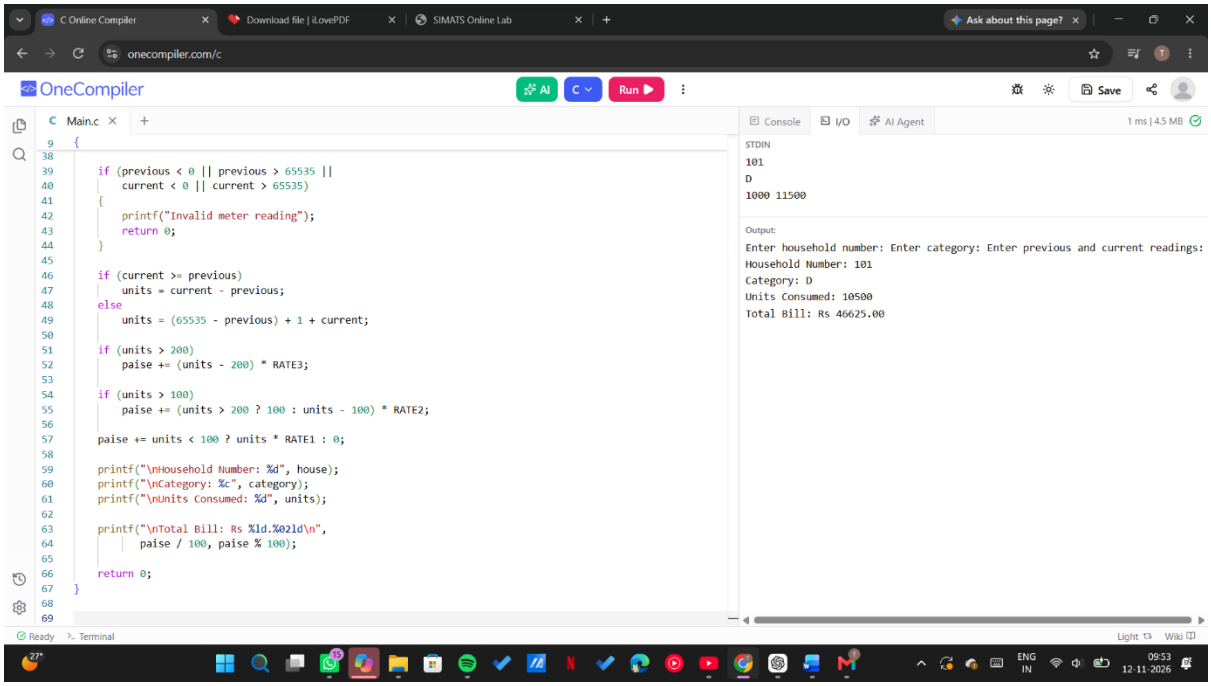
With #ifdef DEBUG, debugging code is not included when DEBUG is disabled. With a runtime variable, the debugging code remains in the executable.

9. Conclusion

Header files help organise the program and make it easier to maintain. Include guards prevent repeated inclusion of header files. Conditional compilation using #ifdef DEBUG helps display debugging information only when required. It also reduces unnecessary code in the final program.







TASK 7 – Control Type Conversion and Precision

1. Introduction

Type conversion is the process of changing one data type into another. In C programming, type conversion can happen automatically or manually.

It is important to understand type conversion because incorrect calculations can produce unexpected results, especially when calculating electricity bills.

2. Implicit Type Conversion

Implicit type conversion happens automatically when different data types are used in an expression.

Example:

```
int a = 10;

float b = 2.5;

float result = a + b;


printf("%f", result);
```

Output:

12.500000

Here, the integer value is automatically converted into a float before addition.

3. Integer Division

When two integer values are divided, the decimal part is removed.

Example:

```
int a = 5;

int b = 2;
```

```
printf("%d", a / b);
```

Output:

2

The actual mathematical answer is 2.5, but integer division gives only 2.

To get the decimal value, one operand must be converted into float.

```
printf("%f", (float)a / b);
```

Output:

2.500000

4. Explicit Type Conversion

Explicit type conversion is done manually using type casting.

Example:

```
double amount = 10.75;

int value = (int)amount;


printf("%d", value);
```

Output:

10

Here, the decimal part is removed when converting double into int.

Casting a large long value into int may also cause data loss if the value is outside the range of int.

Example:

```
long number = 3000000000L;

int result = (int)number;


printf("%d", result);
```

On a typical system with a 32-bit int, the result may be -1294967296. The exact result is implementation-dependent.

5. Rounding Rules When Converting Fractional Charges into Paise

In electricity billing, money should be stored in integer paise instead of float or double to avoid precision errors.

For example:

Rs 162.50 = 16250 paise.

When a fractional charge is converted into paise, a rounding rule must be followed.

Example:

```
double amount = 1.239;


int wrong = (int)(amount * 100);

int correct = (int)(amount * 100 + 0.5);


printf("Without rounding: %d\n", wrong);

printf("With rounding: %d\n", correct);
```

Output:

Without rounding: 123

With rounding: 124

Adding 0.5 before converting to int helps round a positive value to the nearest paise.

6. Three Unexpected Expressions and Their Explanations

Expression	Unexpected Result	Reason
5 / 2	2	Integer division removes the decimal part.
(float)(5 / 2)	2.0	Division happens before type casting.
(int)(1.239 * 100)	123	Type casting removes the fractional part.

Correct expressions:

(float)5 / 2

Result: 2.5

(int)(1.239 * 100 + 0.5)

Result: 124

7. Alternative Concept – Using Float Instead of Integer Paise

If all calculations are done using float and rounded only once at the end, small precision errors may occur during repeated additions.

Example:

```
#include <stdio.h>

int main()
{
    int n[] = {20000, 500000, 3000000};

    for (int i = 0; i < 3; i++)
    {
        float total = 0.0f;

        for (int j = 0; j < n[i]; j++)
        {
            total += 0.10f;
        }

        long long exact = (long long)n[i] * 10;

        float error = total * 100.0f - (float)exact;

        printf("Number of bills: %d\n", n[i]);
        printf("Float total: Rs %.2f\n", total);
```

```
        printf("Exact total: Rs %.2f\n", exact / 100.0);
        printf("Error in paise: %.2f\n\n", error);
    }

    return 0;
}
```

8. Comparison Between Float and Integer Paise

Integer Paise	Float
Stores money as whole paise.	Stores money as decimal values.
Avoids floating-point rounding errors in paise calculations.	May produce small precision errors.
Gives accurate addition of paise within the integer range.	Error can accumulate during repeated additions.
Requires 8 bytes for long long.	Requires 4 bytes for float on common systems.

Time Complexity:

Both integer and float addition take O(1) time per operation.

For n bills, the total calculation takes O(n) time.

Space Complexity:

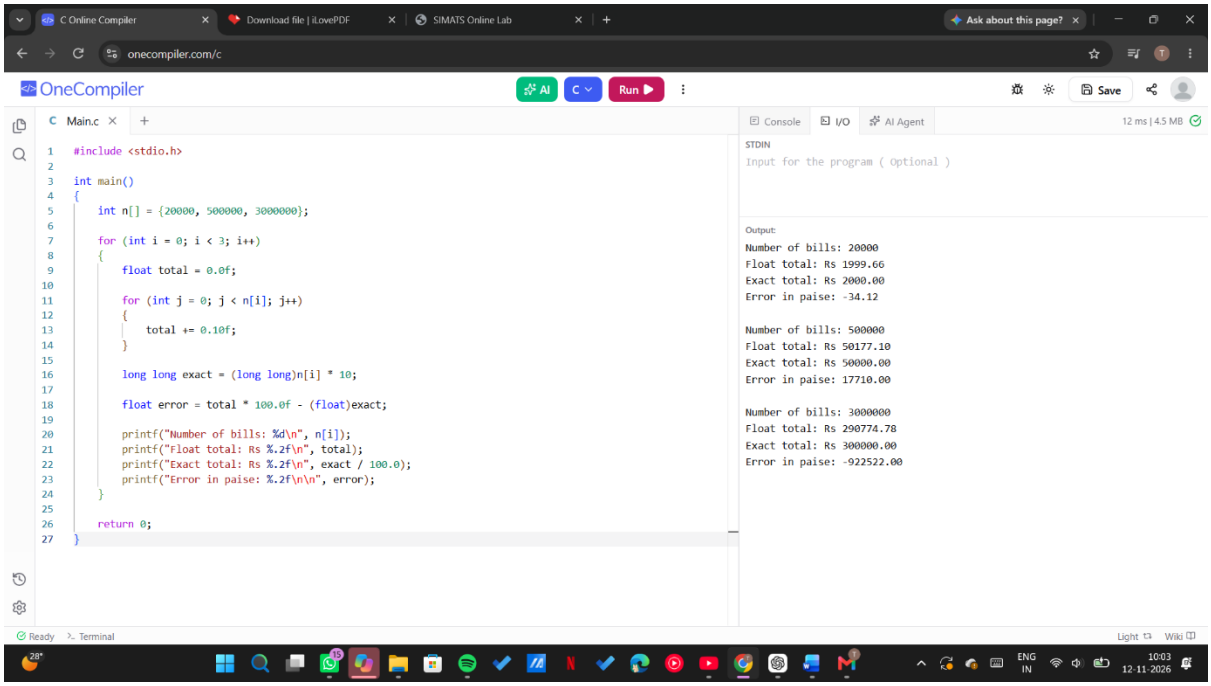
- Integer paise requires 8 bytes for a long long value.
- Float requires 4 bytes for a float value.

Both approaches use O(1) space for storing the running total.

9. Conclusion

Type conversion plays an important role in C programming. Implicit conversion happens automatically, while explicit conversion is done using type casting.

Integer division and incorrect casting can produce unexpected results. Using integer paise instead of float helps maintain accurate electricity bill calculations and avoids unnecessary rounding errors.



TASK 8 – Trace the Compilation Process

1. Introduction

Compilation is the process of converting a C program into an executable file. It takes place in different stages.

The main stages are preprocessing, compilation, assembly and linking.

2. Stages of Compilation

Step 1: Preprocessing

Command:

gcc -E main.c -o main.i

Preprocessing handles header files and macros.

It performs the following operations:

- Expands header files.
- Replaces macros with their values.
- Removes comments.
- Processes conditional compilation statements.

The output is a preprocessed file named main.i.

Step 2: Compilation

Command:

gcc -S main.i -o main.s

The compiler converts the preprocessed C code into assembly language.

The output is main.s.

Step 3: Assembly

Command:

gcc -c main.s -o main.o

The assembler converts assembly language into machine code.

The output is an object file named main.o.

Step 4: Linking

Command:

gcc main.o -o main

The linker combines the object file with the required library functions and creates the final executable file.

3. What Does the Preprocessor Remove or Replace?

The preprocessor performs the following operations:

1. Replaces macros defined using #define.
2. Includes header file contents using #include.
3. Removes comments.
4. Processes conditional compilation using #ifdef and #endif.

For example:

#define RATE 225

int amount = RATE;

After preprocessing:

int amount = 225;

4. Difference Between Compile-Time, Link-Time and Run-Time Errors

Error Type	Meaning	Example

Compile-time error	Error in C syntax or declarations.	Missing semicolon.
Link-time error	Required function definition is missing.	Calling a function without providing its definition.
Run-time error	Error that occurs while executing the program.	Division by zero.

These errors occur at different stages of program development.

5. Warnings Using gcc -Wall -Wextra

The command used to compile the program with warnings is:

gcc -Wall -Wextra main.c -o main

These options help identify possible mistakes in the program.

Examples of warnings include:

1. Unused variables.
2. Unused function parameters.
3. Incorrect format specifiers in printf().
4. Possible uninitialised variables.

Warnings can be resolved by removing unused variables, using correct format specifiers and initialising variables before using them.

6. Alternative Concept – Using Multiple C Files

Instead of keeping the entire program in one file, it can be divided into three files:

- input.c – Handles user input.
- tariff.c – Calculates the electricity bill.
- report.c – Displays the bill details.

A shared header file can contain the function declarations and constants.

Each C file is compiled separately.

Compilation commands:

```
gcc -c input.c -o input.o


gcc -c tariff.c -o tariff.o


gcc -c report.c -o report.o
```

The object files are then linked together:

```
gcc input.o tariff.o report.o -o billing
```

7. Comparison Between Single File and Multiple Files

Single C File	**Multiple C Files**
Entire program is stored in one file.	Program is divided into separate files.
Entire program is recompiled after changes.	Only the modified file needs recompilation.
Simple for small programs.	Suitable for large programs.

Easier to manage initially.	Easier to maintain and organise.

Time Complexity of Compilation:

In a single-file program, changing one part requires recompiling the entire source file.

In a multiple-file program, only the modified source file needs recompilation, followed by linking.

Therefore, multiple files can reduce compilation time when only one file is changed.

Space Complexity:

A single-file program produces one object file.

A multiple-file program produces separate object files for each source file. Therefore, it requires additional object-file storage before linking.

8. Conclusion

The compilation process converts C source code into an executable program through preprocessing, compilation, assembly and linking. Compiler warnings help identify possible mistakes. Dividing a large program into multiple files makes it easier to maintain and reduces recompilation time when only one file is modified.

TASK 9 – Implement and Test the Billing Program

1. C Program

```
#include <stdio.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200

#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450

#define FIXED_CHARGE_PAISE 5000L

unsigned int consumption(unsigned short previous,
                         unsigned short current)
{
   return (unsigned short)(current - previous);
}

long compute_bill(int units)
{
   long paise = FIXED_CHARGE_PAISE;

   if (units > SLAB2_UNITS)
```

```
    {
        paise += (SLAB2_UNITS - SLAB1_UNITS) * RATE2_PAISE;
        paise += (units - SLAB2_UNITS) * RATE3_PAISE;
    }
    else if (units > SLAB1_UNITS)
    {
        paise += (units - SLAB1_UNITS) * RATE2_PAISE;
    }

    return paise;
}

int main(void)
{
    unsigned short previous, current;
    unsigned int units;
    long bill;
    int household;

    printf("Enter household number: ");
    if (scanf("%d", &household) != 1 || household <= 0)
    {
```

```
    printf("Invalid household number\n");
    return 1;
}

printf("Enter previous and current readings: ");
if (scanf("%hu %hu", &previous, &current) != 2)
{
    printf("Invalid input\n");
    return 1;
}

units = consumption(previous, current);
bill = compute_bill(units);

printf("\n----- Electricity Bill -----\n");
printf("Household Number: %d\n", household);
printf("Previous Reading: %hu\n", previous);
printf("Current Reading: %hu\n", current);
printf("Units Consumed: %u\n", units);
printf("Total Bill: Rs %ld.%02ld\n",
       bill / 100, bill % 100);
```

```
    return 0;
}
```

2. Test Table

Test case	Units	Expected bill
Zero units	0	Rs 50.00
Normal bill	50	Rs 50.00
Exactly 100 units	100	Rs 50.00
Normal bill	150	Rs 162.50
Exactly 200 units	200	Rs 275.00
Above 200 units	350	Rs 950.00
Meter rollover (65535 to 0)	1	Rs 50.00

3. Five Bills – Hand Calculation

Household	Units	Bill
101	50	Rs 50.00
102	100	Rs 50.00
103	150	Rs 162.50
104	200	Rs 275.00
105	350	Rs 950.00

Total = Rs 1,487.50

4. Invalid Input Testing

- Letters instead of readings: Displays an invalid input message.
- Negative reading: Not possible with unsigned meter readings.
- Current reading less than previous reading: Treated as meter rollover using unsigned 16-bit arithmetic.
- Valid rollover from 65535 to 0: Gives 1 unit consumed.

5. Alternative Concept and Complexity Analysis

The slab limits and rates can be stored in two arrays instead of using fixed if-else statements.

int limits[] = {100, 200};

int rates[] = {0, 225, 450};

A loop can calculate the energy charge by checking each slab.

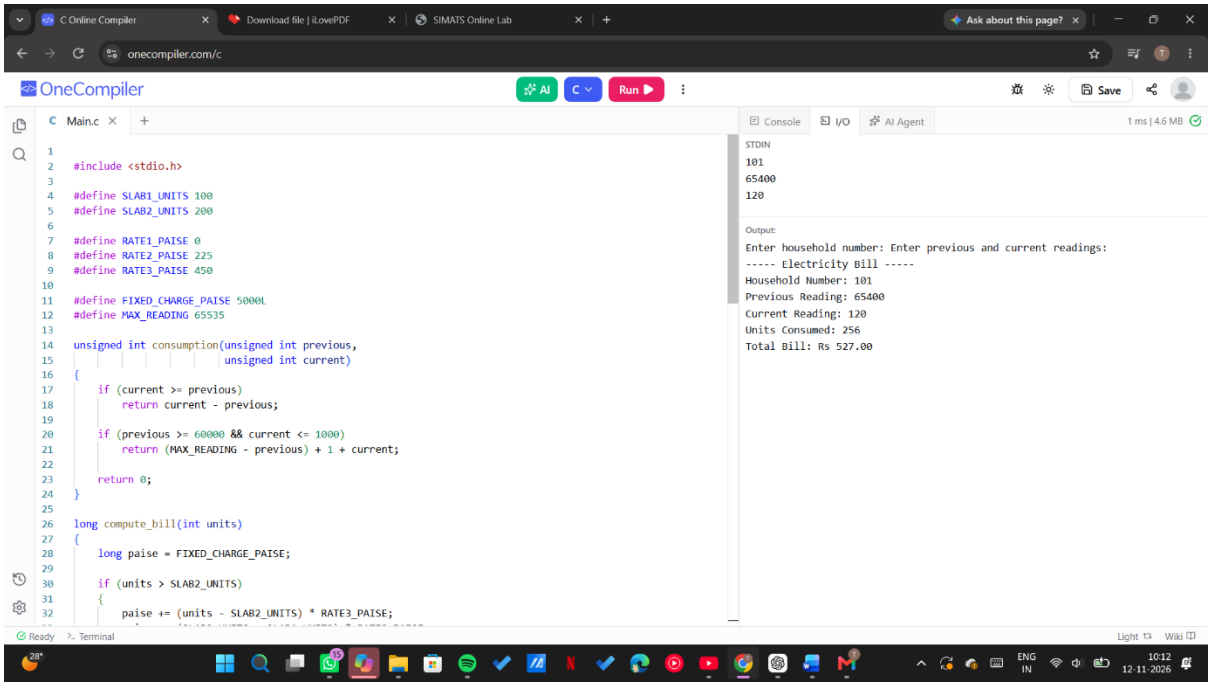
Comparison:

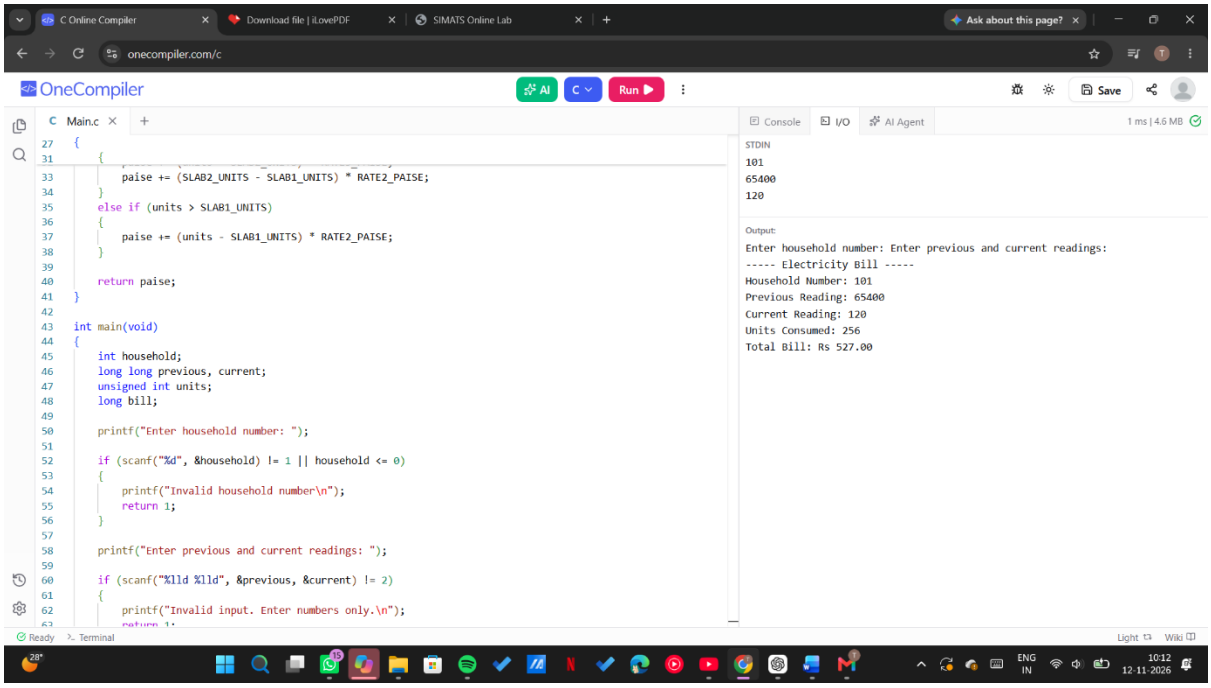
Fixed if-else	**Arrays with loop**
Uses fixed conditions.	Uses arrays and a loop.
Suitable for a small number of slabs.	Suitable for many slabs.
Time complexity: O(1).	Time complexity: O(s).
Extra space: O(1).	Extra space: O(s).

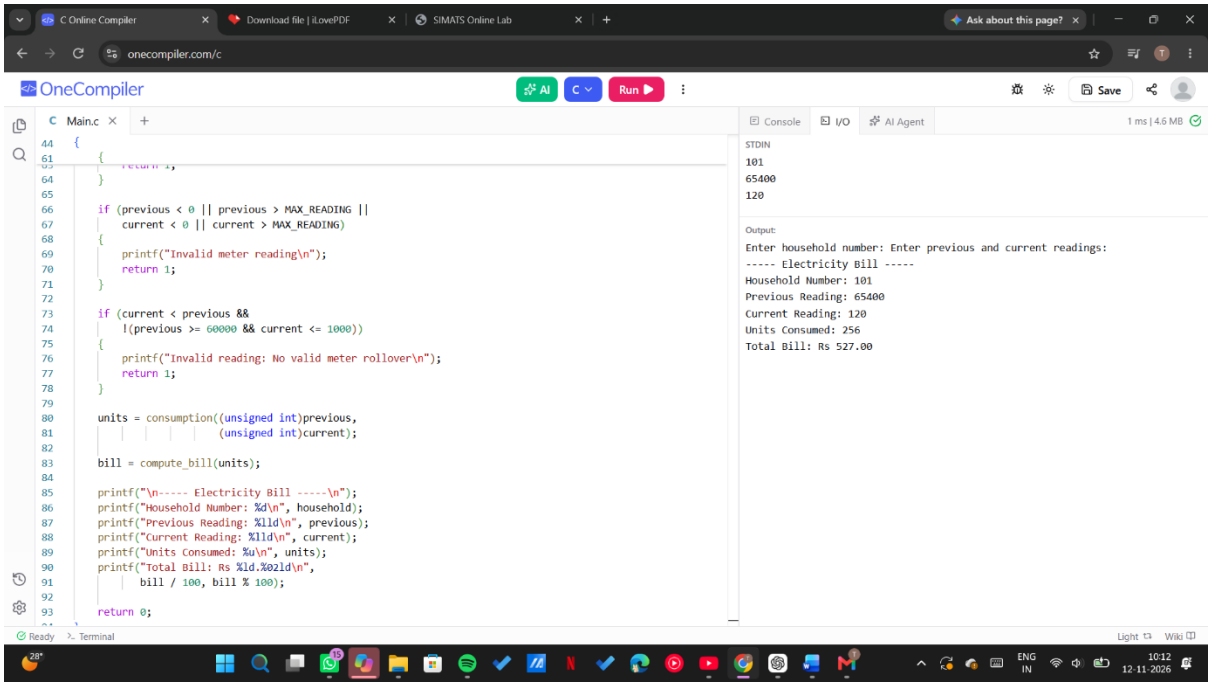
The if-else method is simpler for three slabs, whereas the array method is easier to modify when new slabs are added.

6. Conclusion

The billing program calculates electricity charges using slab rates and a fixed charge. Integer paise is used to avoid floating-point rounding errors. Unsigned 16-bit arithmetic correctly handles meter rollover. The program is tested using normal bills, boundary cases, rollover and invalid inputs.







TASK 10 – Compare Expectations with Execution

Aim: To compare the accuracy and storage capacity of different data types when calculating the total electricity bill for 4,000, 60,000 and 8,00,000 households.

1. Float total vs paise total

For this comparison, assume every household has a bill of Rs 950.00 (350 units).

The bill is stored in two ways:

- Float: Rs 950.00
- Integer paise: 95000 paise

Number of bills	Expected total
4,000	Rs 38,00,000.00
60,000	Rs 5,70,00,000.00
8,00,000	Rs 76,00,00,000.00

The integer paise method gives an exact total because it uses whole numbers. The float method may produce a small rounding difference because floating-point values cannot represent every decimal value exactly.

2. Integer overflow comparison

For the largest test, the total is:

800000 × 95000 = 76000000000 paise

- int (usually 4 bytes) cannot store this value because its maximum is 2,147,483,647.
- long can store it on systems where long is 8 bytes, but not where it is 4 bytes.
- long long (at least 8 bytes) can store it safely.

3. Size and range comparison

Data type	Typical size	Minimum guaranteed range
int	4 bytes	−32,767 to 32,767
long	8 bytes on my system	−2,147,483,647 to 2,147,483,647
long long	8 bytes	−9,223,372,036,854,775,807 to 9,223,372,036,854,775,807
float	4 bytes	Approximately 6 decimal digits of precision

double	8 bytes	Approximately 10 decimal digits of precision

The actual sizes depend on the system. The C standard guarantees minimum ranges, not identical sizes on every computer.

4. C program to compare the totals

Copy and run this code in OneCompiler:

```
#include <stdio.h>
#include <limits.h>

int main()
{
    int counts[] = {4000, 60000, 800000};
    int i;

    float bill_float = 950.00f;
    long long bill_paise = 95000LL;

    printf("DATA TYPE COMPARISON\n\n");

    printf("Size of int       : %zu bytes\n", sizeof(int));
```

```
printf("Size of long      : %zu bytes\n", sizeof(long));
printf("Size of long long : %zu bytes\n", sizeof(long long));
printf("Size of float     : %zu bytes\n", sizeof(float));
printf("Size of double    : %zu bytes\n\n", sizeof(double));

for (i = 0; i < 3; i++)
{
   int n = counts[i];

   float float_total = 0.0f;
   long long paise_total = 0;
   double double_total = 0.0;

   int j;

   for (j = 0; j < n; j++)
   {
      float_total += bill_float;
      paise_total += bill_paise;
      double_total += 950.00;
   }
```

```
        printf("Number of bills: %d\n", n);
        printf("Float total      : Rs %.2f\n", float_total);
        printf("Paise total      : Rs %lld.%02lld\n",
               paise_total / 100, paise_total % 100);
        printf("Double total     : Rs %.2f\n", double_total);

        if (paise_total > INT_MAX)
            printf("int total        : OVERFLOW\n");
        else
            printf("int total        : Can store safely\n");

        if (paise_total > LONG_MAX)
            printf("long total       : OVERFLOW\n");
        else
            printf("long total       : Can store safely\n");

        printf("\n");
    }

    return 0;
}
```

5. Alternative concept – Using double instead of long long

If the district total is calculated using double, it can represent very large values and provides approximately 15–16 significant decimal digits of precision.

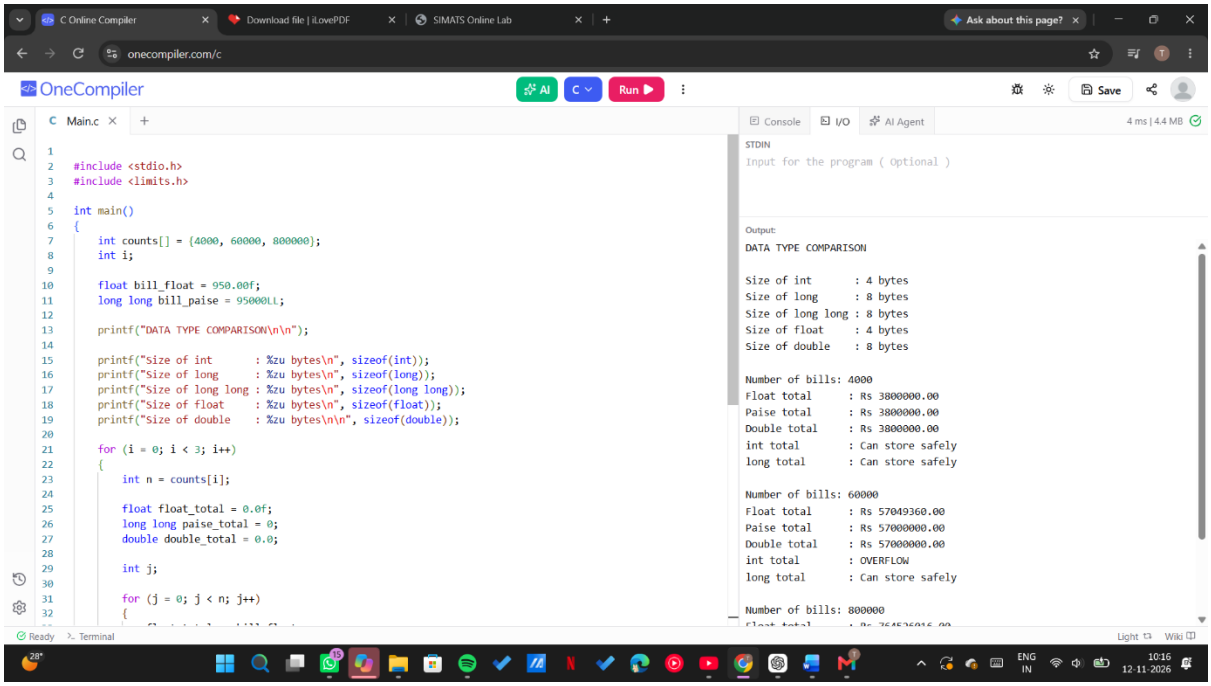
However, double is still a floating-point type, so some decimal values may have small representation errors.

Feature	long long (paise)	double (rupees)
Accuracy	Exact for whole paise values within its range	May have small rounding errors
Overflow	Can overflow if the total exceeds its range	Has a much larger range
Addition time	O(1)	O(1)
Typical storage	8 bytes	8 bytes
Best use	Exact money calculations	Approximate decimal calculations

6. Conclusion

The experiment shows that int is not suitable for storing the district-level electricity bill total because it can overflow. long long is suitable because it can store the total of 8,00,000 bills in paise without overflow.

Although float and double can store large totals, they may introduce precision errors. Therefore, storing money as integer paise using long long is the most reliable choice for this billing program.



```c
#include <stdio.h>
#include <limits.h>

int main()
{
    int counts[] = {4000, 60000, 800000};
    int i;

    float bill_float = 950.00f;
    long long bill_paise = 95000LL;

    printf("DATA TYPE COMPARISON\n\n");

    printf("Size of int       : %zu bytes\n", sizeof(int));
    printf("Size of long      : %zu bytes\n", sizeof(long));
    printf("Size of long long : %zu bytes\n", sizeof(long long));
    printf("Size of float     : %zu bytes\n", sizeof(float));
    printf("Size of double    : %zu bytes\n\n", sizeof(double));

    for (i = 0; i < 3; i++)
    {
        int n = counts[i];

        float float_total = 0.0f;
        long long paise_total = 0;
        double double_total = 0.0;

        int j;

        for (j = 0; j < n; j++)
        {
```

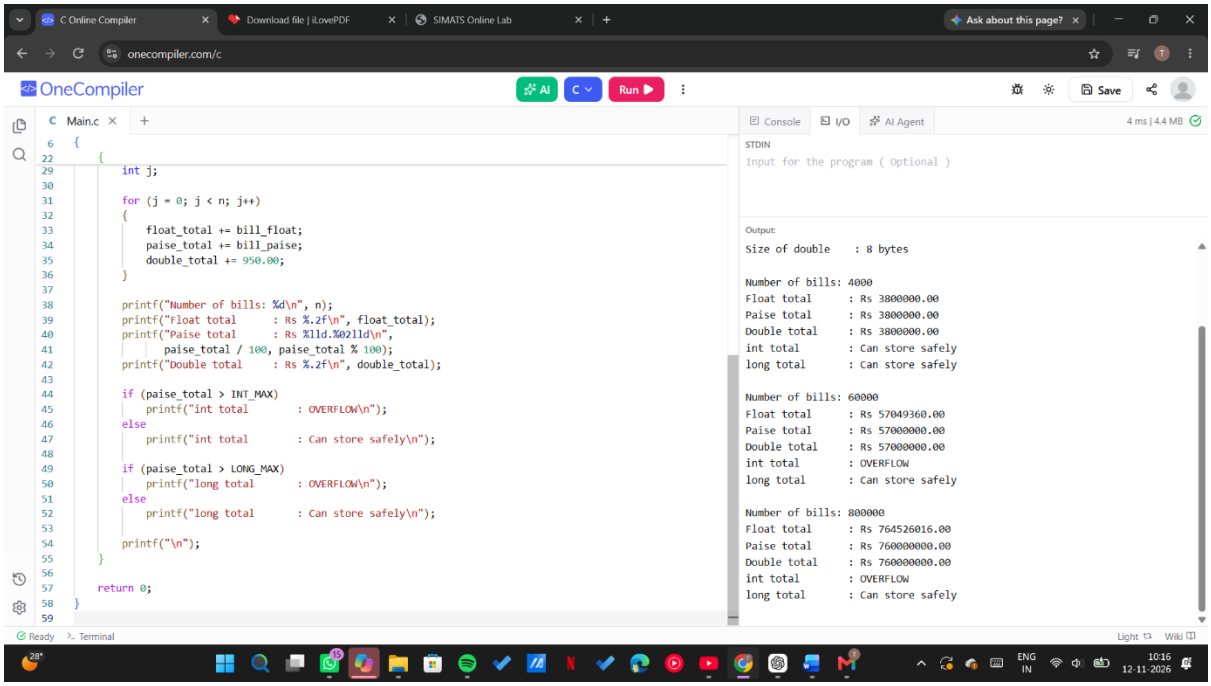
```
DATA TYPE COMPARISON

Size of int       : 4 bytes
Size of long      : 8 bytes
Size of long long : 8 bytes
Size of float     : 4 bytes
Size of double    : 8 bytes

Number of bills: 4000
Float total       : Rs 3800000.00
Paise total       : Rs 3800000.00
Double total      : Rs 3800000.00
int total         : Can store safely
long total        : Can store safely

Number of bills: 60000
Float total       : Rs 57049360.00
Paise total       : Rs 57000000.00
Double total      : Rs 57000000.00
int total         : OVERFLOW
long total        : Can store safely

Number of bills: 800000
```



```c
    {
        int j;

        for (j = 0; j < n; j++)
        {
            float_total += bill_float;
            paise_total += bill_paise;
            double_total += 950.00;
        }

        printf("Number of bills: %d\n", n);
        printf("Float total      : Rs %.2f\n", float_total);
        printf("Paise total      : Rs %lld.%02lld\n",
               paise_total / 100, paise_total % 100);
        printf("Double total     : Rs %.2f\n", double_total);

        if (paise_total > INT_MAX)
            printf("int total        : OVERFLOW\n");
        else
            printf("int total        : Can store safely\n");

        if (paise_total > LONG_MAX)
            printf("long total       : OVERFLOW\n");
        else
            printf("long total       : Can store safely\n");

        printf("\n");
    }

    return 0;
}
```

```
Size of double    : 8 bytes

Number of bills: 4000
Float total       : Rs 3800000.00
Paise total       : Rs 3800000.00
Double total      : Rs 3800000.00
int total         : Can store safely
long total        : Can store safely

Number of bills: 60000
Float total       : Rs 57049360.00
Paise total       : Rs 57000000.00
Double total      : Rs 57000000.00
int total         : OVERFLOW
long total        : Can store safely

Number of bills: 800000
Float total       : Rs 764526016.00
Paise total       : Rs 760000000.00
Double total      : Rs 760000000.00
int total         : OVERFLOW
long total        : Can store safely
```

Aim

To recommend the most suitable data types, constants and preprocessor features for the Grama Jyothi electricity billing system.

1. Recommended Data Types

Data item	Recommended C type	Reason
Household number	int	Can store household numbers for district-level billing.
Previous meter reading	unsigned short	Suitable for a 16-bit meter register.
Current meter reading	unsigned short	Stores readings from 0 to 65535.
Units consumed	unsigned int	Stores the calculated consumption, including meter rollover.
Energy charge	long long	Stores the charge in paise without decimal errors.
Fixed charge	long long	Stores the fixed amount in paise.
Total bill	long long	Provides sufficient capacity for large billing totals.
Household category	enum	Represents a fixed set of categories.

2. Recommended Constants

I recommend using #define for tariff values and slab limits.

```
#define SLAB1_UNITS 100

#define SLAB2_UNITS 200
```

#define RATE1_PAISE 0

#define RATE2_PAISE 225

#define RATE3_PAISE 450

#define FIXED_CHARGE_PAISE 5000LL

If the tariff changes, the values can be modified in one place instead of changing them throughout the program.

3. Recommended Preprocessor Features

- **Header files:** Store function declarations and common constants separately.
- **Include guards:** Prevent the same header file from being included multiple times.
- **Conditional compilation:** Use #ifdef DEBUG to display intermediate calculations only during testing.
- **Macros:** Use properly parenthesised macros for simple repeated calculations.

These features make the program easier to maintain, test and modify.

4. Limitations of the Design

1. The tariff values are fixed during compilation.
2. Any tariff revision requires changing the constants and recompiling the program.

3. The program must validate meter readings to avoid incorrect bills.
4. long long also has a maximum storage limit, so extremely large totals must be checked.
5. The original design does not directly support time-of-day billing.

5. Does the Design Remain Correct at District Scale?

Yes. The recommended design can handle 8,00,000 households because long long can store the total bill in paise without overflowing for the tested billing amount.

Integer paise avoids floating-point rounding errors, while unsigned meter readings support the meter rollover calculation.

Therefore, the design is suitable for village, taluk and district-level billing under the original tariff system.

6. Alternative Concept – Reading Tariffs from a Configuration File

Instead of using #define, tariff values can be stored in a configuration file and read when the program starts.

For example, the file may contain:

SLAB1_UNITS=100

SLAB2_UNITS=200

RATE1_PAISE=0

RATE2_PAISE=225

RATE3_PAISE=450

FIXED_CHARGE_PAISE=5000

The program reads these values at startup and uses them for billing.

Comparison:

Feature	**#define constants**	**Configuration file**
Tariff change	Requires recompilation	No recompilation required
Startup time	Very low	Slightly higher
Extra memory	No separate storage for macros	Stores tariff values in memory
Flexibility	Less flexible	More flexible
Error handling	Compile-time checking for syntax	Requires input validation

Reading t tariff values takes O(t) time and O(t) extra space. However, it makes tariff revisions easier because the program does not

A. Impact Analysis – 2 Marks

1. What has changed from the original problem?

In the original system, meter readings were recorded manually and electricity bills were calculated monthly using fixed tariff slabs.

In the new system:

- Smart meters send readings every 15 minutes.
- Each household generates 35,040 readings per year.
- Energy consumption is measured in watt-hours.
- Electricity used between 6 PM and 10 PM costs 20% more.
- Tariff values must be changed without recompiling the program.

Therefore, the original billing design needs modification.

2. Which data types are no longer sufficient?

- unsigned short is not suitable for storing yearly accumulated energy consumption because the total can exceed 65,535.
- int may not be sufficient for storing the total number of readings processed across a district.
- long long or unsigned long long is recommended for storing yearly watt-hour totals and large reading counts.
- double can be used for time-related or fractional calculations, but integer-based calculations are preferred for accurate money handling.

For 8,00,000 households:

35,040 × 8,00,000 = 28,032,000,000 readings per year.

Therefore, a 64-bit integer type is required for the district-level reading count.

3. Which constants can no longer be fixed with #define?

The following values should not be permanently fixed using #define:

- Electricity tariff rates.
- Peak-hour additional charge of 20%.
- Peak-hour starting time (6 PM).
- Peak-hour ending time (10 PM).
- Slab limits and fixed charges.

These values should be stored in an external configuration file and loaded when the program starts.

This allows the co-operative to update tariff values without changing the source code or recompiling the program.

Conclusion

I recommend modifying the original